Data Analysis using SQL

Executive summary -

Context

This report analyzes transactional data from Amazon Brazil's e-commerce platform across customer behavior, payment trends, product performance, and seasonal sales patterns.

Scope

The analysis covers orders, payments, products, customers, and sellers' data using SQL across three analytical modules.

Key Takeaways

- Credit cards dominate all payment segments - highest average value and highest order share. Boleto is the second highest. Vouchers and debit cards remain underutilised.
- 97% of customers are first-time buyers - repeat purchases are extremely low, making a structured loyalty or re-engagement program a priority.
- Peak sales fall in March, April, and May. Summer and Spring drive the most revenue, while September and October are the weakest months.
- 614 products have missing category names - this directly affects product discovery and category-level reporting accuracy and needs to be fixed at the data entry level.
- Top 20 customers average 64,918 BRL per order — a high-value segment worth targeting with an exclusive rewards program.

ANALYSIS PART 1

1. To simplify its financial reports, Amazon India needs to standardize payment values. Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.

   Output: payment_type, rounded_avg_payment

Query | Query History

```
--Analysis part 1
--A1 Q1.Round the average payment values to integer (no decimal) for each payment type
--and display the results sorted in ascending order
--Output: payment_type, rounded_avg_payment

select payment_type , round (avg ( payment_value)) as rounded_avg_payment
from payments
group by payment_type
order by rounded_avg_payment asc;
```

- Observations:
- The average payment value is higher for credit card transactions, followed by boleto & debit card, with the lowest average payment value from vouchers.

2. To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type. Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order

   Output: payment_type, percentage_orders

```sql
-- A1 Q2.Calculate the percentage of total orders for each payment type,
--rounded to one decimal place, and display them in descending order
--Output: payment_type, percentage_orders

select payment_type, round (count (distinct order_id)*100
/ (select count(distinct order_id) from payments), 1) as percentage_orders
from payments
group by 1
order by 2 desc;
```

- Observations:
- The percentage of total orders was seen to be higher from credit card transactions, followed by boleto, with the least being voucher and debit card.
- The average payment value and percentage of total orders is seen to be higher from credit card transactions.

3. Amazon India seeks to create targeted promotions for products within specific price ranges. Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.

   Output: product_id, price

```sql
--A1 Q3. Identify all products priced between 100 and 500 BRL that contain
--the word 'Smart' in their name. Display these products, sorted by price in
--descending order. Output: product_id, price

select oi.product_id , oi.price :: int
from order_items oi join products p
on oi.product_id = p.product_id
where lower(p.product_category_name) like '%smart%' and oi. price between 100 and 500
order by oi.price desc;
```

- Observations:
- As per the results, there are a total of 34 products with the word ‘smart’ in their product name, and prices ranged between 100 and 500 BRL.
- Among which 50% of the products range between 100 and 200 BRL.

4. To identify seasonal sales patterns, Amazon India needs to focus on the most successful months. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

   Output: month, total_sales

```
--A1 Q4. Determine the top 3 months with the highest total sales value,
--rounded to the nearest integer. Output: month, total_sales

select to_char(o.order_purchase_timestamp, 'month' ) as month ,
round (sum (p.payment_value),0) as total_sales
from orders o join payments p
on o. order_id = p. order_id
group by 1
order by 2 desc
limit 3;
```

- Observations:
- The top 3 months with the highest total sales value are found to be May, August, and July.

5. Amazon India is interested in product categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.

   Output: product_category_name, price_difference

```
--A1 Q5. Find categories where the difference between the maximum and minimum
--product prices is greater than 500 BRL.Output: product_category_name, price_difference

select p. product_category_name ,
round (max (oi. price) - min (oi.price)) as price_difference
from products p join order_items oi on p.product_id = oi.product_id
group by 1 having max(oi.price) - min(oi.price) > 500;
```

- Observations:
- The data shows a total of 57 products listed, which have a price difference of > 500 BRL from the maximum price and minimum price value.

6. To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts. Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.

   Output: payment_type, std_deviation

```sql
--A1 Q6.  Identify the payment types with the least variance in transaction amounts,
--sorting by the smallest standard deviation first. Output: payment_type, std_deviation

select payment_type , round (stddev (payment_value),2) as std_deviation
from payments group by payment_type
order by 2 asc;
```

- Observations:
- Voucher payment type has the lowest standard deviation compared to other payment types & has the most consistent transaction amounts.
- Customers have bought almost the same value vouchers during the transaction.

7. Amazon India wants to identify products that may have an incomplete name in order to fix it from their end. Retrieve the list of products where the product category name is missing or contains only a single character.

   Output: product_id, product_category_name

```sql
-- A1 Q7. Retrieve the list of products where the product category name is missing or
--contains only a single character. Output: product_id, product_category_name

select product_id , product_category_name
from products
where product_category_name is null
or length(product_category_name) = 1;
```

- Observations :
- A total of 614 products have their product category name missing.

ANALYSIS PART 2

1. Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high). Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count

   Output: order_value_segment, payment_type, count

```
--Analysis 2
--A2Q1. Segment order values into three ranges: orders less than 200 BRL, between 200
--and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these
--ranges and display the results in descending order of count
--Output: order_value_segment, payment_type, count

select case
when payment_value <200 then 'Low'
when payment_value between 200 and 1000 then 'Medium'
else 'High'
end as order_value_segment, payment_type ,
count (*) as count from payments
group by 1 , 2
order by count desc;
```

-

- Observations :
- All 3 segments have higher payments coming through ‘credit card’ followed by ‘boleto’.
- The highest count of transactions has been made for ‘Low’, followed by the ‘Medium’ segment range.

2. Amazon India wants to analyse the price range and average price for each product category. Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.

   Output: product_category_name, min_price, max_price, avg_price

```sql
--A2Q2. Calculate the minimum, maximum, and average price for each category, and list them
--in descending order by the average price. Output: product_category_name, min_price,
--max_price, avg_price

select p.product_category_name , round (min (oi.price),2) as min_price,
round (max (oi.price),2) as max_price , round (avg (oi.price),2) as avg_price
from products p join order_items oi
on p.product_id = oi.product_id
group by 1
order by avg_price desc;
```

- Observations:
- A total of 79 product categories are listed with price ranging from ~ 12BRL to ~ 6729BRL.

3. Amazon India wants to identify the customers who have placed multiple orders over time. Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.

   Output: customer_unique_id, total_orders

```sql
--A2Q3. Find all customers with more than one order, and display their customer unique IDs
--along with the total number of orders they have placed. Output: customer_unique_id, total_orders

select c. customer_unique_id , count (o. order_id) as total_orders
from orders o join customer_data c on
c.customer_id = o.customer_id
group by c.customer_unique_id
having count (o.order_id)> 1 order by 2 desc;
```

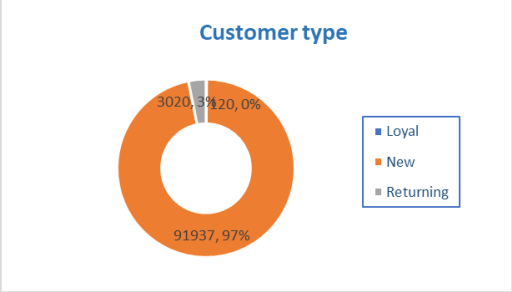
- Observations-
- A total of 3140 customers have placed more than 1 order.
- The highest no. of orders placed is 16.

4. Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' –order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history. Use a temporary table to define these categories and join it with the customers table to update and display the customer types.

   Output: customer_unique_id, customer_type

```
--A2Q4.  Amazon India wants to categorize customers into different types ('New - order qty. = 1' ;
--'Returning' -order qty. 2 to 4;  'Loyal' - order qty. >4) based on their purchase history.
--Use a temporary table to define these categories and join it with the customers table to update
--and display the customer types. Output: customer_unique_id, customer_type

with customer_total_orders as ( select c.customer_unique_id , count ( distinct o. order_id)
as total_orders
from customer_data c join orders o
on c. customer_id = o. customer_id
group by 1)
select ct. customer_unique_id,
case
when total_orders = 1  then 'New'
when total_orders >=2 and total_orders <=4 then 'Returning'
else 'Loyal' end as customer_type from customer_total_orders ct
order by customer_type;
```



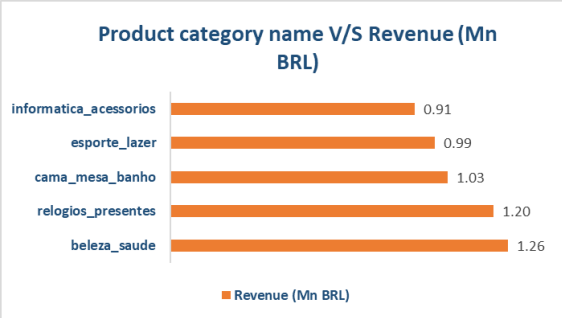
- Observations:
- 97% of customers come from ‘New’segment.

5. Amazon India wants to know which product categories generate the most revenue. Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.

   Output: product_category_name, total_revenue

```sql
--A2Q5. Amazon India wants to know which product categories generate the most revenue. Use joins
--between the tables to calculate the total revenue for each product category. Display the top 5
--categories. Output: product_category_name, total_revenue

select p.product_category_name , round (sum (oi.price),2) as total_revenue
from products p join order_items oi
on p.product_id = oi.product_id
group by product_category_name
order by total_revenue desc
limit 5;
```



- Top 5 product category names with the highest revenue.

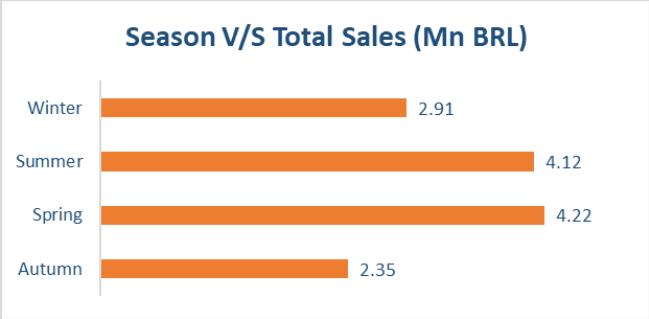
Analysis 3

1. The marketing team wants to compare the total sales between different seasons. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.

   Output: season, total_sales

```sql
--A3 Q1. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter)
--based on order purchase dates, and display the results. Spring is in the months of March, April
--and May. Summer is from June to August and Autumn is between September and November and rest
--months are Winter. Output: season, total_sales

with cte as ( select extract ( month from o. order_purchase_timestamp) as month ,
sum ( oi.price ) as total_sales  from orders o join order_items oi
on o.order_id = oi.order_id
group by 1)
select ( case
when month in (3 , 4 , 5) then 'Spring'
when month in (6, 7, 8) then 'Summer'
when month in (9, 10, 11) then 'Autumn'
else 'Winter' end ) as season, round ( sum (total_sales ), 2) as total_sales from cte
group by season;
```



- Observations:

  The summer and spring seasons have the highest revenue.

2. The inventory team is interested in identifying products that have sales volumes above the overall average. Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

   Output: product_id, total_quantity_sold

```
--A3 Q2. The inventory team is interested in identifying products that have sales volumes above
--the overall average. Write a query that uses a subquery to filter products with a total quantity
--sold above the average quantity. Output: product_id, total_quantity_sold

select product_id , count (order_id) as total_quantity_sold
from order_items
group by 1
having count(order_id) > (select avg (total_quantity) from (select count(order_id) as
total_quantity from order_items group by product_id ) a);
```

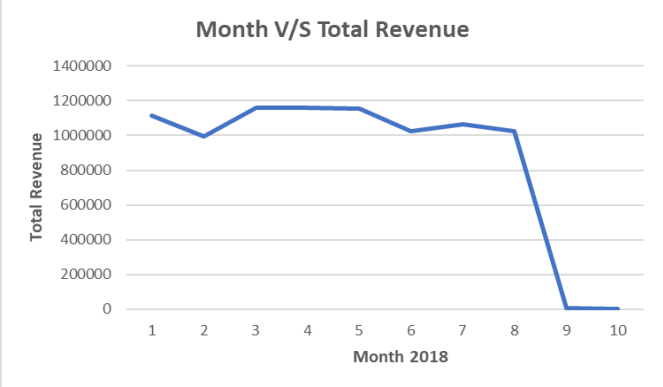
- Observation:
- One of the products was sold in the highest quantity of 527.

3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018). Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.

   Output: month, total_revenue

```sql
--A3 Q3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue
--trends over the past year (year 2018). Run a query to calculate total revenue generated each month
--and identify periods of peak and low sales. Export the data to Excel and create a graph to
--visually represent revenue changes across the months. Output: month, total_revenue

select
extract (month from o.order_purchase_timestamp) as month,
round ( sum(p.payment_value), 2) as total_revenue
from orders o join payments p  on o. order_id = p. order_id
where extract (year from o.order_purchase_timestamp) = 2018
group by 1;
```



- Observations:
- The total revenue peaked in the months of April , March & May and the lows were seen in September and October.

4. A loyalty program is being designed for Amazon India. Create a segmentation based on purchase frequency: ‘Occasional’ for customers with 1-2 orders, ‘Regular’ for 3-5 orders, and ‘Loyal’ for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.

   Output: customer_type, count

```sql
--A3 Q4.Create a segmentation based on purchase frequency: ‘Occasional’ for customers with 1-2
--orders, ‘Regular’ for 3-5 orders, and ‘Loyal’ for more than 5 orders. Use a CTE to classify
--customers and their count and generate a chart in Excel to show the proportion of each segment.
--Output: customer_type, count

with cte as
( select customer_id, count ( order_id ) as num_of_orders
from orders
group by 1 )
select
( case when num_of_orders <= 2 then 'Occasional'
when num_of_orders >=3 and num_of_orders <=5 then 'Regular'
else 'loyal' end ) as customer_type , count ( num_of_orders ) as count from cte
group by 1;
```

5. Amazon wants to identify high-value customers to target for an exclusive rewards program. You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.

   Output: customer_id, avg_order_value, and customer_rank

```sql
--A3 Q5. You are required to rank customers based on their average order value (avg_order_value)
--to find the top 20 customers. Output: customer_id, avg_order_value, and customer_rank

with cte as
( select o. customer_id ,
round ( sum ( p.payment_value ) / count ( distinct o. order_id ) , 2 )
as avg_order_value
from orders o join payments p
on o.order_id = p.order_id
group by 1)
select
customer_id , avg_order_value ,
dense_rank () over (order by avg_order_value desc) as customer_rank
from cte limit 20;
```

- Observations:
- The overall average order value coming from the top 20 customer accounts for 64918.42 Mn BRL

6. Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.

   Output: product_id, sale_month, and total_sales

```sql
--A3 Q6. Amazon wants to analyze sales growth trends for its key products over their lifecycle.
--Calculate monthly cumulative sales for each product from the date of its first sale.
--Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.
--Output: product_id, sale_month,total_sales

with recursive cte1 as (
select oi. product_id , date_trunc ('month', min (o.order_purchase_timestamp))::
date as sale_month,
date_trunc ( 'month', max (o.order_purchase_timestamp)):: date as max_month
from order_items oi
join orders o on o.order_id = oi.order_id
group by oi.product_id

union all

select product_id , (sale_month + interval '1 month'):: date, max_month
from cte1 where sale_month < max_month ), monthly_actual_sales as
( select oi.product_id , date_trunc ( 'month',o.order_purchase_timestamp):: date as sale_month,
sum (oi.price)as actual_sales from order_items oi join orders o on o.order_id = oi.order_id
group by 1, 2)
```

```sql
select ct.product_id , to_char (ct.sale_month , 'yyyy - mm') as sale_month,
round (sum(coalesce(mas.actual_sales,0))over
( partition by ct.product_id order by ct.sale_month)) as total_sales
from cte1 ct
left join monthly_actual_sales mas on ct.product_id = mas.product_id and ct.sale_month = mas.sale_month
order by ct.product_id , ct.sale_month;
```

7. To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018). Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.

   Output: payment_type, sale_month, monthly_total, monthly_change.

```sql
--A3 Q7. Amazon wants to compute the total sales for each payment method and calculate the
--month-over-month growth rate for the past year (year 2018). Write query to first calculate total
--monthly sales for each payment method, then compute the percentage change from the previous month.
--Output: payment_type, sale_month, monthly_total, monthly_change.

with monthly_totals as ( select p.payment_type,
extract(YEAR  FROM o.order_purchase_timestamp) as sale_year,
extract(MONTH FROM o.order_purchase_timestamp) as sale_month,
round(SUM(p.payment_value)::numeric, 2)as monthly_total
from payments p
join orders o on p.order_id = o.order_id
where extract(year from o.order_purchase_timestamp) = 2018
group by p.payment_type,
extract(year from o.order_purchase_timestamp),
extract(month from o.order_purchase_timestamp)
)
select payment_type, sale_month, monthly_total,
round((monthly_total - lag(monthly_total) over (partition by payment_type order by sale_year,
sale_month))
 / nullif(lag(monthly_total) over (partition by payment_type order by sale_year, sale_month), 0)
 * 100, 2 )as monthly_change
from monthly_totals
order by payment_type, sale_year, sale_month;
```